Final Exam Practice Questions

ECO 4971/6971 – Machine Learning and Econometrics

Solutions

Introduction to Statistical Learning
Classes 5–13 (cumulative)

NOTE: These solutions were generated by AI. Use at your own risk - they may be incorrect or incomplete.

Section 1: Core Concept Questions (By Class)

1.1 Review: Classes 5–8B (Fundamentals, Linear Regression, KNN, Classification)

1. A colleague claims that "because this model has the lowest training error, it must be the best model we have tried." Briefly explain why this reasoning is flawed and what you would look at instead.

   Solution. Training error can always be driven down by making the model more flexible (more predictors, higher polynomial degree, smaller K in KNN, deeper trees), even when the extra flexibility is just fitting noise. What matters for future predictions is *test error*, estimated via a held-out test set or cross-validation, not training error. The "best" model is the one with the lowest *out-of-sample* error.

2. In a multiple linear regression of `Sales` on `TV`, `Radio`, and `Newspaper`, the coefficient on `Radio` is 0.19. State in one sentence what this coefficient means to a marketing manager who is not a statistician.

   Solution. Holding TV and newspaper spending fixed, each additional unit spent on radio advertising is associated with an additional 0.19 units of sales, on average. (The phrase "holding TV and newspaper fixed" is the heart of multiple-regression interpretation – it is what distinguishes a partial effect from a marginal correlation.)

3. Define the **residual** for observation i, and write the expression that ordinary least squares (OLS) is minimizing. One line each.

   Solution. The residual is $e_i = y_i - \hat{y}_i$ (observed minus fitted). OLS chooses coefficients to minimize the residual sum of squares

$$\text{RSS} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2.$$

4. Confusion matrix: 140 / 10 / 20 / 30. Compute (a) overall accuracy, (b) precision for class 1, and (c) recall for class 1.

   Solution. Total $n = 200$. (a) Accuracy $= (140 + 30)/200 = 170/200 = \mathbf{0.85}$. (b) Precision for class 1 $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FP}) = 30/(30 + 10) = \mathbf{0.75}$. (c) Recall for class 1 $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FN}) = 30/(30 + 20) = \mathbf{0.60}$. The asymmetry (precision > recall) means that when the model predicts "1" it is usually right, but it misses 40% of true positives.

5. A student says, "Adding predictors always improves a regression model." Comment on the validity of this statement in-sample and out-of-sample.

   Solution. *In-sample (training):* the statement is essentially true for OLS – adding a predictor can never *increase* training RSS, since the larger model contains the smaller one as a special case. So R^2 weakly increases and training MSE weakly decreases, mechanically.
   Out-of-sample (test): the statement is false. Each new predictor adds variance to the estimator. If the new variable is mostly noise, test error *rises* even though training error fell. This is exactly the bias–variance tradeoff. The right diagnostic is held-out test MSE, cross-validated MSE, or a complexity-penalized criterion (adjusted R^2, AIC, BIC).

6. Explain why a high correlation between two predictors can make coefficient interpretation unstable, even when prediction accuracy remains good.

   Solution. When two predictors carry nearly the same information, the regression cannot cleanly attribute the joint effect between them: many different $(\hat{\beta}_1, \hat{\beta}_2)$ pairs fit the data almost equally well, so small changes in the sample swing the individual coefficients (sometimes flipping their signs) and inflate their standard errors. Predictions, however, depend on the *combined* contribution $\hat{\beta}_1 X_1 + \hat{\beta}_2 X_2$, which is well-identified – so the model can still predict well even when the individual coefficients are unreliable for inference.

7. In one sentence each, define **training set**, **validation set**, and **test set**.

   Solution. *Training set:* the data used to *fit* the model (estimate parameters). *Validation set:* a held-out subset used to *tune* hyperparameters or compare candidate models. *Test set:* a final held-out subset used *once*, after all modeling decisions are made, to obtain an honest estimate of generalization error.

8. Why is it risky to repeatedly tune many models using the same test set?

   Solution. Each time you peek at the test set and adjust your model in response, you are implicitly fitting to it – it stops being independent of your modeling choices. After many such "adaptive" decisions, the test error you report is biased downward (too optimistic) because your final model has been selected to look good on *that particular* test set. The test set should be touched once, at the end. Cross-validation or a separate validation set is the right tool for iterative tuning.

9. Give one realistic situation in which a simpler linear model could be preferred over a more accurate black-box classifier.

   Solution. Several acceptable answers:

- *Regulatory / explainability:* a bank denying credit must justify each decision to the applicant – a logistic regression with signed coefficients does this directly, while a black-box ensemble does not.
- *Inference:* an economist wants to estimate the partial effect of a policy variable, not to forecast.
- *Small data:* with n only modestly larger than p, the linear model has lower variance and may actually predict better than a flexible model that overfits.
- *Production / engineering cost:* a linear model is cheap to deploy, audit, and monitor for drift.

Any *one* of these is sufficient.

10. In the statistical learning setup $Y = f(\mathbf{X}) + \varepsilon$, explain the roles of $f(\mathbf{X})$ and ε. Which part can a better algorithm reduce, and which part cannot be predicted from $\mathbf{X}$?

    Solution. $f(\mathbf{X})$ is the systematic, learnable relationship between predictors and response. ε is the irreducible error: random or unobserved variation that is, by construction, independent of $\mathbf{X}$. A better algorithm can only improve our *estimate* $\hat{f}$ of f – thereby reducing the *reducible* error – but it cannot predict ε from $\mathbf{X}$, no matter how flexible. The irreducible error sets a floor on attainable test MSE.

11. A regression table reports a 95% confidence interval for β_1 of $[0.02, 0.08]$. What does this interval suggest about the sign and statistical significance of the relationship?

    Solution. The interval lies entirely above zero, so the relationship is *positive* (a one-unit increase in X_1 is associated with somewhere between 0.02 and 0.08 more of Y). Because 0 is not in the 95% CI, we reject $H_0 : \beta_1 = 0$ at the 5% level: the effect is statistically significant. (Note: significance does not say anything about whether the effect is *practically* large – that depends on the units and context.)

12. Why do we include only $K - 1$ dummy variables for a categorical predictor with K categories when the regression includes an intercept?

    Solution. If we included all K dummies, they would sum to a column of ones – the same as the intercept column – creating perfect multicollinearity (the "dummy variable trap"). The design matrix would be singular and OLS would not have a unique solution. Dropping one category breaks the collinearity; the omitted category becomes the *reference*, and the coefficients on the included dummies are interpreted as the difference between each category and that reference, with the reference's mean absorbed into the intercept.

13. A KNN regression with $K = 3$ finds nearest-neighbor responses 12, 15, and 18. What prediction does KNN make?

    Solution. KNN regression averages the neighbors' responses:

    $$\hat{y} = (12 + 15 + 18)/3 = \mathbf{15}.$$

    For classification, KNN takes a majority vote instead.

14. Why is feature scaling especially important for KNN? Use the idea of distance in your explanation.

    Solution. KNN identifies neighbors using a distance metric (usually Euclidean) on the predictor space. If one variable is measured on a vastly larger numeric scale than the others – say income in dollars vs. age in years – the squared differences in income will dominate the distance computation purely because of units, even if income is no more informative than age. Standardizing each predictor (subtract mean, divide by SD, or min–max scale) puts all predictors on a comparable footing so that "nearest" reflects substantive similarity rather than measurement units.

15. For KNN, compare a very small value of K with a very large value of K using bias and variance language.

    Solution. Small K (extreme case $K = 1$) is highly flexible: each prediction is shaped by a single neighbor, so the decision surface is jagged. This gives *low bias* (it can match almost any local pattern) but *high variance* (predictions move a lot if the training data shifts). Large K averages over many neighbors, producing a smooth, almost constant predictor: *high bias* (real local structure is washed out) but *low variance* (predictions are stable across resamples). The optimal K trades these off and is chosen by cross-validation.

1.2 Class 9: Resampling Methods (Cross-Validation & Bootstrap)

16. Distinguish between a model's **parameters** and its **tuning parameters**, and give one example of each from a method we covered this term.

    Solution. *Parameters* are estimated from training data by optimizing a loss – e.g., the regression coefficients $\hat{\beta}_j$ in OLS, or the split thresholds inside a decision tree. *Tuning parameters* (hyperparameters) are choices that govern *model complexity* and must be set *before* fitting – e.g., the polynomial degree, K in KNN, the penalty λ in ridge/lasso, the depth of a tree, or the learning rate in boosting. Tuning parameters are selected by validation or cross-validation, not by minimizing training error.

17. Describe the **validation set approach** in one or two sentences. What is the main drawback of using a single random train/validation split to choose a model?

    Solution. Randomly split the data once into a training set (used to fit candidate models) and a validation set (used to estimate their test error), then pick the model that performs best on the validation set. The drawback is that the estimate of test error depends heavily on which observations happen to land in the validation set: a different random split can change both the estimated error and the chosen model – especially with smaller samples. The estimate is also biased upward because the model is fit on only a fraction of the data.

18. Compare **leave-one-out cross-validation (LOOCV)** and **k-fold cross-validation**. Give one reason we usually prefer k-fold in practice.

    Solution. LOOCV holds out one observation at a time and refits the model n times; k-fold partitions the data into k roughly equal folds and refits k times. k-fold is preferred because (i) it is far cheaper computationally ($k = 5$ or 10 vs. n fits), and (ii) its estimate of test error typically

has *lower variance* than LOOCV. The n training sets in LOOCV are nearly identical – they differ by only one observation – so their predictions are highly correlated and their average has high variance; k-fold training sets overlap less, so the fold predictions are less correlated.

19. In a bootstrap procedure with $B = 1{,}000$ replications of a simple linear regression, we obtain 1,000 estimates of the slope $\hat{\beta}_1$. Explain in one sentence how we use these to obtain a **standard error** for $\hat{\beta}_1$.

    Solution. The bootstrap standard error is simply the sample standard deviation of the 1,000 bootstrap estimates of $\hat{\beta}_1$:

$$\mathrm{SE}_{\mathrm{boot}}(\hat{\beta}_1) = \sqrt{\tfrac{1}{B-1}\sum_{b=1}^{B}\left(\hat{\beta}_1^{(b)} - \overline{\hat{\beta}_1}\right)^2}.$$

    A 95% percentile confidence interval is the 2.5th and 97.5th percentiles of those same 1,000 values.

1.3 Class 10: Model Selection (Subset Selection, Ridge, Lasso)

20. Explain in words what **best subset selection** does when we have p candidate predictors. Why does it become infeasible for large p? What cheaper alternative did we discuss?

    Solution. Best subset selection fits *every* possible model – all 2^p subsets of the predictors – and picks the best one using a criterion such as cross-validation error, AIC, BIC, or adjusted R^2. The number of models grows exponentially in p, so it becomes infeasible once p is moderate (already $\sim$1 million for $p = 20$). The cheaper alternative is *forward (or backward) stepwise selection*, which greedily adds (or drops) one predictor at a time. It evaluates only $O(p^2)$ models and is not guaranteed to find the global best, but it usually does well in practice.

21. Write down the **ridge regression** objective function. How does ridge regression differ from ordinary least squares?

    Solution. Ridge minimizes

$$\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 + \lambda\sum_{j=1}^{p}\beta_j^2,$$

    i.e., the OLS residual sum of squares *plus* an ℓ_2 penalty on the (non-intercept) coefficients. Compared to OLS, ridge trades a small increase in bias for a (potentially much larger) reduction in variance by shrinking coefficients toward zero. The shrinkage is controlled by λ: when $\lambda = 0$, ridge collapses to OLS; as $\lambda \to \infty$, all coefficients are pushed toward zero.

22. The **lasso** replaces the ridge penalty with an ℓ_1 penalty. State one important practical consequence of this change when we have many candidate predictors.

    Solution. The ℓ_1 penalty $\lambda\sum_j|\beta_j|$ shrinks some coefficients *exactly* to zero at sufficiently large λ. So the lasso performs *automatic variable selection* and produces a sparse, more interpretable model – valuable when most candidates are irrelevant or when stakeholders want a short list of important predictors. Ridge, by contrast, shrinks coefficients smoothly but never sets them to exactly zero.

23. Why must predictors be **standardized** before fitting ridge or lasso regression? What could go wrong if we forget to do this?

    Solution. The penalty applies the same λ to every coefficient, but the magnitude of a coefficient depends on the scale of its predictor. A variable measured in dollars will have a tiny coefficient (so a small penalty contribution) compared to one measured in counts – purely because of units, not because of importance. Without standardization, the choice of measurement units silently determines which variables get shrunk most. Standardizing each predictor to mean zero and unit variance makes the penalty comparable across variables.

24. When we choose λ for ridge or lasso, we typically use cross-validation rather than training-set fit. Explain briefly why.

    Solution. Training-set fit always improves as $\lambda \to 0$: at $\lambda = 0$ ridge/lasso reduce to OLS, which by definition minimizes training RSS. So training error can never tell us when to *stop* shrinking – it would always pick $\lambda = 0$. Cross-validation estimates *out-of-sample* error at each candidate λ, which has a real minimum, and we pick the λ at that minimum (or at the "one-standard-error" rule for a slightly more parsimonious model).

1.4 Class 11: Tree-Based Methods (Trees, Random Forests, Boosting)

25. A **regression tree** partitions the predictor space by choosing, at each step, a variable and a cutpoint. What criterion does the algorithm use to pick the split, and why is the procedure called **greedy**?

    Solution. At each node, the algorithm picks the (variable, cutpoint) pair that produces the largest immediate reduction in residual sum of squares (RSS) on the current partition. It is *greedy* because it optimizes locally – the best split right now – without looking ahead to whether a slightly worse split now would enable better splits later. As a result, the resulting tree is rarely the globally optimal tree, but greedy splitting is computationally tractable while exhaustive search is not.

26. For a **classification tree**, we split using a criterion such as the **Gini index** or **entropy** rather than RSS. In one sentence each, explain what these two measures are trying to capture.

    Solution. Both are measures of *node impurity* – how mixed the class labels are within a region. The *Gini index*, $\sum_k \hat{p}_k(1 - \hat{p}_k)$, is the probability that two randomly drawn observations from the node belong to different classes (small when one class dominates). *Entropy*, $-\sum_k \hat{p}_k \log \hat{p}_k$, measures the information content (disorder) of the class distribution and is zero only when the node is pure. Splits are chosen to reduce whichever impurity measure is used.

27. What is **bagging**? Why does averaging the predictions of many trees tend to reduce variance? What assumption about the individual trees matters here?

    Solution. Bagging (*bootstrap aggregating*) fits many trees on bootstrap resamples of the training data and averages their predictions (or takes a majority vote for classification). Averaging reduces variance because the random idiosyncrasies of different trees – caused by the random resamples – partially cancel. The crucial assumption is that the trees are not too *correlated*: if all trees end up looking nearly identical (e.g., because one predictor dominates every tree's first

split), their average has essentially the same variance as a single tree, and bagging buys you nothing. This is exactly the problem random forests were invented to fix.

28. How does a **random forest** differ from plain bagging? Specifically, explain the role of the parameter m (the number of predictors considered at each split), and why the default $m \approx \sqrt{p}$ often helps.

    Solution. At each split, a random forest randomly samples m of the p predictors and considers *only* those as candidates for that split. With $m = p$ the forest reduces to bagging, and if one or two predictors dominate, every bagged tree splits on them first and the trees look almost identical. Restricting candidates to a small random subset forces the trees to use *different* variables, which *decorrelates* them; the average of decorrelated trees has much lower variance. The rule-of-thumb default $m \approx \sqrt{p}$ for classification (and $p/3$ for regression) reliably produces this decorrelation across a wide range of datasets, though it can be tuned.

29. What is the **out-of-bag (OOB) error**, and why does it give us a "free" estimate of test error without a separate validation set?

    Solution. Each bootstrap sample uses about two-thirds of the observations, leaving roughly one-third *out of bag* for each tree. For any observation i, we can predict it using only the trees that did *not* see i during training – so for that observation, those trees behave like an honest test fit. Averaging this error over all observations gives the OOB error. Because each observation is effectively held out for some trees, OOB error estimates test error without us needing to set aside a separate validation set.

1.5 Class 12: Deep Learning (Feedforward Networks)

30. In a feedforward neural network, each hidden unit computes an **activation function** applied to a weighted sum of its inputs. Why is the activation function essential? What happens to a deep network if we replace every activation function with the identity function $g(z) = z$?

    Solution. The activation function introduces *nonlinearity*, which is what lets a network represent complicated, non-additive relationships between inputs and outputs. Without it – if $g(z) = z$ everywhere – every layer is just a linear transformation $z \mapsto Wz + b$, and the composition of any number of linear transformations is itself linear. A network of *any depth* would collapse to a single linear model: no more expressive than plain linear regression, no matter how many layers or units it has.

31. Give two distinct strategies we discussed for reducing **overfitting** in a neural network, and state in one sentence how each works.

    Solution. Any *two* of the following:

    - *Dropout:* at each training step, randomly "drop" a fraction of neurons; this forces the network not to rely on any single unit and behaves like averaging many smaller subnetworks (an ensemble effect).
    - *Early stopping:* monitor validation loss during training and stop when it starts rising, before the network begins memorizing training noise.

- ℓ_2 *weight decay:* add a penalty on the squared weights to the loss, analogous to ridge regression; this shrinks weights and reduces variance.
- *Data augmentation / more data:* expand the effective training set so the model has less room to memorize idiosyncrasies.
- *Smaller architecture:* reduce depth or width to lower the parameter count.

32. Why do very deep and very wide neural networks typically require **large datasets** to perform well? Phrase your answer in terms of the bias–variance tradeoff.

    Solution. A large network has very low *bias* – it can represent almost any function, including the truth – but also very high *variance*, because it has many parameters that can fit idiosyncratic noise in any single training sample. Large datasets reduce that variance: with many observations the noise effectively averages out and the network settles on the signal rather than the noise. On a small dataset, the same architecture has too much flexibility relative to the signal-to-noise ratio and overfits badly, so simpler methods (regularized linear models, tree ensembles) typically win.

1.6 Class 13: Unsupervised Learning (PCA, K-Means, Hierarchical Clustering)

33. What distinguishes **unsupervised** learning from **supervised** learning? Give one concrete example of a question each is designed to answer.

    Solution. Supervised learning has a target Y and tries to predict it from features X – e.g., "given a borrower's credit history, will they default?" Unsupervised learning has only features X and tries to discover *structure* or *patterns* – e.g., "do our customers fall into natural groups?" (clustering) or "what low-dimensional summary of 50 features captures most of the variation?" (PCA). There is no ground-truth Y to validate against in the unsupervised case, which is why model evaluation is fundamentally harder.

34. Describe the K**-means clustering** algorithm in three short steps. What does the algorithm try to minimize? Why might we run the algorithm multiple times from different random starts?

    Solution. (1) Randomly initialize K cluster centers. (2) *Assignment step:* assign each observation to the nearest center (Euclidean distance). (3) *Update step:* move each center to the mean of the observations currently assigned to it. Repeat (2)–(3) until assignments no longer change. The algorithm minimizes the total within-cluster sum of squared distances $\sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \mu_k\|^2$. Because the optimization is greedy and non-convex, it can get stuck in a local minimum that depends on the random initialization, so we re-run from many random starts (`n_init` in scikit-learn) and keep the solution with the lowest within-cluster variance.

35. Explain what a **dendrogram** shows in hierarchical clustering. How do we use it to choose a number of clusters?

    Solution. A dendrogram is a tree diagram of the merge sequence in agglomerative clustering: each leaf is one observation, and pairs of leaves/clusters are joined at a height equal to the dissimilarity at which they were merged. To get k clusters, draw a horizontal line that crosses exactly k vertical branches; the connected subtrees below the line define the clusters. A nice

property is that the entire tree is built once, and you can choose k *after* looking at the structure – often by cutting at a height where merges suddenly become much more expensive (a long vertical gap).

36. Name two different **linkage** methods (e.g., complete, single, average) used in hierarchical clustering, and state in one sentence how they differ.

    Solution. Linkage defines the distance between two clusters from the pairwise distances of their members. *Complete linkage* uses the *maximum* pairwise distance (compact, balanced clusters). *Single linkage* uses the *minimum* distance (long, stringy "chained" clusters). *Average linkage* uses the *mean* distance (a compromise). In practice, complete and average are the most common defaults; single linkage is sensitive to noise.

37. Why should the variables typically be **standardized** before running PCA? Illustrate with a quick example.

    Solution. PCA finds directions of maximum *variance*, but variance depends on the units of measurement. If income is in dollars (variance in the billions) and "number of children" is a count (variance around 2), income will completely dominate PC1 purely because its numeric variance is astronomically larger – even though the count variable might be equally informative. Standardizing each variable to mean 0 and SD 1 puts all variables on an equal footing, so PCA reflects *patterns of co-variation*, not units of measurement.

Section 2: Output-Based Questions (Graphs & Regression Output)

1. **Ridge coefficient path.** (a) What happens to the coefficients as λ increases? (b) Does ridge regression ever set a coefficient exactly to zero on this plot? Why/why not?

   Solution. (a) As λ increases, all the standardized coefficients shrink *smoothly* toward zero. At small λ they are close to the OLS estimates; at very large λ they all approach zero together.
   (b) No, none hits zero exactly. The ridge penalty $\lambda \sum_j \beta_j^2$ is differentiable everywhere and curves smoothly through the origin, so the penalized first-order conditions keep coefficients nonzero – they only *asymptote* to zero. This is the geometric reason ridge does not perform variable selection; lasso, with its non-differentiable ℓ_1 penalty, does.

2. **Lasso coefficient path.** (a) Describe what happens to individual coefficients as λ grows large. (b) Which predictors are the *last* to be shrunk to zero, and what does that suggest about their relationship with `Balance`?

   Solution. (a) Unlike ridge, each lasso coefficient is shrunk toward zero and then *flatlines at exactly zero* once λ exceeds a predictor-specific threshold. Less informative predictors drop out first; only a handful survive at large λ.
   (b) On the `Credit` data, the last predictors to be eliminated are typically `Rating`, `Limit`, and `Student`, with `Rating` surviving longest. This suggests these three carry the strongest *partial* signal for `Balance` after controlling for the others – credit limit/rating naturally scale with how much balance a customer carries, and being a student systematically shifts balance levels.

3. **Regression tree (depth 2).** (a) What log salary would this tree predict for a player with 6 years of experience and 120 hits? (b) Which predictor appears more important at the *top* of the tree, and why does that ordering matter?

   Solution. (a) Years $= 6 > 4.5$, then Hits $= 120 > 117.5$, so the player falls in the rightmost terminal leaf with predicted log salary $\approx$ **6.74** (so predicted salary $\approx e^{6.74} \approx$ \$846K).
   (b) The root split is on `Years`, so `Years` is identified as the most informative single predictor overall. The top split matters because everything below it is conditional on it – the entire downstream structure of the tree (and the implicit interactions it represents) is shaped by that one greedy choice. The ordering of variables down the tree is roughly an importance ranking for the fitted tree.

4. **Random forest feature importance (Boston, target medv).** (a) Which two features drive the most predictions? (b) A student concludes that "`lstat` *causes* housing prices to fall." Are they correct and why?

   Solution. (a) The top two are `rm` (average rooms per dwelling) and `lstat` (% lower-status population); together they typically account for ~80% of the total importance.
   (b) No. Feature importance measures how much a variable contributes to *predictive accuracy within this particular fitted model* – it is not a causal estimate. `lstat` is correlated with many unmeasured confounders (school quality, neighborhood amenities, infrastructure, historical patterns of segregation), and the importance plot cannot separate these. The plot would tell you to *include* `lstat` if you want predictions, but causal claims need a research design that handles confounding (instruments, natural experiments, randomization), not a predictive ranking.

5. **Neural network training curves.** (a) Roughly at which epoch would **early stopping** have stopped training? (b) What is happening to the network after that point, and how could we reduce this behavior?

   Solution. (a) The validation loss reaches its minimum somewhere around **epoch 25–35**; early stopping would halt training at the epoch where validation loss begins to rise consistently.
   (b) After that point, training loss keeps falling while validation loss rises – the classic signature of *overfitting*: the network is memorizing training-set idiosyncrasies it cannot generalize. Standard fixes are early stopping (stop at the validation minimum we just identified), dropout, ℓ_2 weight decay, reducing the network's size, data augmentation, or collecting more training data.

6. **PCA scree plot (USArrests).** (a) How many principal components would you retain, and what criterion did you use? (b) If PC1 has large positive loadings on `Murder`, `Assault`, and `Rape`, what interpretation would you give to a state's score on PC1?

   Solution. (a) **Two** components is the standard answer here. PC1 explains $\approx 62\%$ and PC2 $\approx 25\%$ of the variance, jointly $\approx 87\%$, and the scree plot shows a clear "elbow" between PC2 and PC3. (You could justify $k = 1$ on a strict elbow rule, or $k = 3$ on a $\geq 90\%$ cumulative-variance rule – both are defensible if the reasoning is consistent.)
   (b) PC1 acts as an "overall violent-crime" axis: a high PC1 score means a state with high levels of violent crime in general (across murder, assault, and rape simultaneously); a low PC1 score means a low-violent-crime state.

7. **Dendrogram (USArrests, complete linkage, dashed cut).** (a) How many clusters would be produced if we cut the tree at the height of the dashed line? (b) If we lowered the cut, would the number of clusters increase or decrease, and why?

   Solution. (a) The dashed line at height ≈ 4 intersects four vertical branches, so cutting there produces **4 clusters**.
   (b) Lowering the cut *increases* the number of clusters. A lower horizontal line is below more merge events, so clusters that had been glued together at higher heights split back apart – and there are more standing branches to count. Conversely, raising the cut produces fewer, larger clusters.

8. **Heart confusion matrix (depth-3 tree, test set).** The matrix shows TN = 36, FP = 12, FN = 14, TP = 28. Compute (a) accuracy, (b) precision for class "AHD", and (c) recall for class "AHD".

   Solution. Total $n = 36 + 12 + 14 + 28 = 90$.
   (a) Accuracy $= (36 + 28)/90 = 64/90 \approx \mathbf{0.711}$. (b) Precision for AHD $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FP}) = 28/(28 + 12) = 28/40 = \mathbf{0.700}$. (c) Recall for AHD $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FN}) = 28/(28 + 14) = 28/42 \approx \mathbf{0.667}$. The model is slightly better at not over-flagging "AHD" than it is at catching every true case – so it leaves a meaningful number of patients with disease undetected, which is the more costly type of error in a medical setting.

9. **ROC comparison on Heart test set.** (a) Which model has better threshold-free discrimination, and what visual evidence supports this? (b) Can a model with better AUC still have worse accuracy at a fixed threshold of 0.5? Explain briefly.

Solution. (a) The two models are essentially tied, with logistic regression slightly ahead: AUC ≈ 0.923 for logistic regression vs. ≈ 0.915 for the random forest. The visual evidence is that the logistic curve sits at or just above the random-forest curve over most of the false-positive-rate range. (Either model would be a reasonable answer here; the key point is to read off the AUCs and note how close they are.)
(b) Yes. AUC measures discrimination across *all* thresholds – it asks "how often does the model rank a true positive above a true negative?" Accuracy at threshold 0.5 depends on both calibration and the class balance. A model can have higher AUC (better ranking) but predict probabilities that are systematically too low or too high near 0.5, so flipping the threshold matters. Two models with the same AUC can have very different accuracies at a fixed threshold.

10. **Lasso CV curve, Boston.** (a) What does the dashed line represent? (b) Why do we usually choose λ using CV rather than selecting the smallest training error?

    Solution. (a) The dashed vertical line marks the value of $\log(\lambda)$ that minimizes the 10-fold CV MSE – the "chosen alpha" (here approximately $\log \lambda \approx -2$, i.e., $\lambda \approx 0.14$). Beyond that point, MSE starts climbing as the lasso shrinks too aggressively.
    (b) Training error is monotonically minimized as $\lambda \to 0$ (no shrinkage at all – ordinary least squares), so it would always pick $\lambda = 0$ and never tell us when to start regularizing. CV estimates *out-of-sample* error, which has a true minimum at the right amount of shrinkage and lets us pick a λ that generalizes. (You may also see a "one-standard-error rule" line that picks a slightly larger λ for parsimony.)

11. **Random forest OOB error vs. number of trees.** (a) Describe the pattern of OOB error as trees are added. (b) What explains the initial decrease in OOB error? (c) Why does error typically plateau rather than keep dropping?

    Solution. (a) OOB error starts high with very few trees (here ≈ 0.40 at one tree), drops sharply over the first 20–30 trees, and then plateaus at ≈ 0.13–0.14 from ~ 50 trees onward.
    (b) The initial decrease is the variance-reduction effect of bagging: averaging the predictions of many bootstrap-trained, decorrelated trees cancels out idiosyncratic errors of individual trees. With only one tree, there is nothing to average; with 30 trees, the noise has mostly washed out.
    (c) After enough trees, the average converges – adding more trees does not change the law-of-large-numbers limit. The remaining error reflects *bias* of the tree ensemble (the kinds of relationships individual trees can represent) plus *irreducible noise*, neither of which more trees can fix. Adding trees beyond the plateau costs compute without buying accuracy; this is why $\sim$ 200–500 trees is a typical default.

12. **Training and test error vs. polynomial degree (1–15).** (a) Which degree appears to minimize test MSE? (b) Why does training MSE keep falling after that point?

    Solution. (a) Test MSE is minimized at about polynomial degree **3** (you would also accept degree 2 if the curves are essentially flat between them; the key point is that the minimum is at low degree). Beyond that, test MSE flattens or rises.
    (b) Training MSE is monotonically non-increasing in degree because higher-degree polynomials are *nested*: anything a degree-3 polynomial can fit, a degree-4 polynomial can fit at least as well (just zero out the new coefficient). So training error must weakly decrease as flexibility grows – but the extra flexibility past degree 3 is fitting noise, not signal, so test error stops improving.

13. **KNN tuning on Advertising.** (a) Which K would you choose? (b) What would likely happen if $K = 1$ were chosen instead?

    Solution. (a) Pick the K at the minimum of the test-MSE curve – here approximately $K = 4$ (a small flat region near $K = 3$–5 is also defensible).
    (b) $K = 1$ uses only the single nearest training point for each prediction, so the model has *very low bias but very high variance*: the prediction surface zig-zags through every training point and overreacts to noise. Test MSE is typically much worse than at moderate K. (For classification, $K = 1$ is also notoriously sensitive to mislabeled training points.)

14. **Logistic probability curve (top-quartile balance vs. credit limit).** (a) Around what credit limit does the model assign probability 0.5? (b) If the threshold were raised from 0.5 to 0.7, what would happen to false positives and false negatives?

    Solution. (a) Read off the credit limit at which the curve crosses $p = 0.5$ – approximately **6,400** for this fit.
    (b) Raising the threshold makes the classifier more *conservative* about predicting "high balance." Fewer customers cross the higher bar, so:

    - *False positives* (low-balance predicted as high) decrease,
    - *False negatives* (high-balance predicted as low) increase.

    This is the standard precision/recall tradeoff – raising the threshold raises precision and lowers recall.

15. **Lasso CV curve, Credit.** (a) What does the vertical dashed line represent? (b) Why should λ be chosen using CV rather than training error?

    Solution. (a) The dashed vertical line marks the λ that minimizes 10-fold CV MSE (here $\log \lambda \approx -0.92$, so $\lambda \approx 0.4$).
    (b) Because training MSE is monotonically minimized at $\lambda = 0$ (no penalty = ordinary least squares), training error favors no shrinkage at all and cannot identify a useful penalty. CV estimates out-of-sample error, which has a true minimum at the "right" amount of shrinkage and lets us pick a λ that generalizes. (Same logic as the Boston curve in Q47.)

16. **Hitters regression tree (depth 3).** (a) Trace the tree to predict log salary for a player with 8 years and 90 hits. (b) Why might a deeper tree overfit this dataset?

    Solution. (a) Following the splits for Years $= 8 > 4.5$ and then Hits $= 90$ leads to a terminal leaf with predicted log salary $\approx$ **6.12**.
    (b) A deeper tree creates ever smaller leaves, each averaging only a handful of players. With $\sim$260 observations in `Hitters`, those small leaves capture accidental patterns (a few unusual players) rather than stable relationships. Predictions become very sensitive to which players happen to be in the training set, so test error rises even though training RSS keeps falling. Pruning, an explicit max_depth, or growing a random forest instead are the standard ways to control this.

Section 3: Challenging Questions

1. In the **geometric view** of shrinkage, ridge and lasso both constrain the coefficient vector to lie inside a region around the origin. What is the difference between the shapes of the constraint regions, and how does this difference affect the coefficients?

   Solution. Both methods can be viewed as minimizing RSS subject to the coefficient vector lying inside a constraint region. The contours of RSS are ellipses; the optimum is the point where the smallest RSS ellipse first touches the constraint region.

   - *Lasso (ℓ_1):* the constraint region is a *diamond* (in 2-D) or "cross-polytope" (higher dimensions), with *corners on the coordinate axes.* RSS ellipses have a strong tendency to first touch a corner – and a corner on an axis means at least one coefficient is exactly zero. Hence the lasso produces *sparse* solutions and performs variable selection.
   - *Ridge (ℓ_2):* the constraint region is a *circle* (or hypersphere) – smooth, with no corners. The first contact point is essentially always somewhere on the rim where *both* coefficients are nonzero. Coefficients shrink smoothly toward zero but never reach it.

   This corner/no-corner geometry is the entire reason lasso does variable selection and ridge does not.

2. A random forest reports nearly identical **OOB error** and **held-out test error**. Explain briefly why this agreement is expected. Under what circumstances might the two differ?

   Solution. For each observation, OOB predictions come from the $\sim 1/3$ of trees that did not see it during training – so OOB error is, in effect, an average of predictions from trees trained on data *excluding* that observation. This is conceptually the same thing a held-out test set does, so the two should agree closely when the training and test data come from the same distribution. They can diverge when (i) the test set comes from a *different* distribution (covariate shift, time change, new geography), (ii) the training set is small enough that few trees see any given OOB observation – making OOB noisy – or (iii) the test set is small and gives a noisy benchmark.

3. Two configurations for gradient boosting: (A) $\lambda = 0.01$, $B = 2{,}000$ trees; (B) $\lambda = 0.3$, $B = 50$ trees. Which is more likely to **overfit** if B is not tuned carefully, and why?

   Solution. (B) – large learning rate with few trees – is more likely to overfit when B is chosen poorly. Each individual step moves the ensemble a long way, so even a handful of trees can chase noise, and the validation-error vs. B curve has a sharp minimum that is easy to overshoot. (A), with a small learning rate, makes many tiny corrections; overfitting sets in only much later, and the validation curve has a broad, gentle minimum that is easy to land near. The practical rule of thumb – small λ with large B, both chosen by CV – comes from exactly this logic.

4. In a binary classification setting with very imbalanced classes (2% positives), logistic regression and a random forest both achieve 98% accuracy. Why may this be *uninformative*, and what resampling-based diagnostic could help?

   Solution. A trivial classifier that predicts "negative" for everyone already gets 98% accuracy on 2% positives, so "98%" tells you almost nothing about how either model handles the minority class. You need metrics that condition on the positive class: *precision*, *recall*, *F1*, ROC–AUC,

and especially the area under the precision–recall curve (PR-AUC), which is more sensitive to the rare class than ROC-AUC.
For a resampling diagnostic, use *stratified k-fold cross-validation* (preserving the 2% positive rate in every fold) to compute these metrics across folds, and compare the distributions for the two models. A single fold can look great by luck on a rare event; CV gives you fold-to-fold variability so you can tell whether the difference is real.

5. A colleague insists that cross-validation is unnecessary "because we have a huge dataset — we can just split once into train and test." Under what conditions is this defensible? Give one concrete scenario where you would still recommend k-fold CV even with a large sample.

   Solution. The colleague is roughly right when (i) the sample is so large that even a single split leaves both train and test sets individually huge, so the test estimate has tiny variance, and (ii) you are not making many adaptive modeling decisions. In that case CV mostly buys compute time.
   You would still use k-fold CV when:

   - the model is sensitive to which observations are in the training set (rare events, natural groups, time series),
   - you are comparing many candidate models and want tight error bars to separate close competitors (single-split error has higher variance),
   - you need to tune hyperparameters on something other than the final test set,
   - the model is cheap to refit, so CV is essentially free.

   "Big data" does not erase the value of repeated estimates of test error.

6. Suppose a random forest has much lower training error than a single tree but similar test error. Give two possible explanations.

   Solution. Several defensible explanations; *any two*:

   - The single tree was already capturing most of the stable signal, so the forest's extra flexibility mainly improves the in-sample fit (variance reduction does not translate into much new test improvement when the bias was already small).
   - The dataset is dominated by *irreducible noise*: there is a floor on test error that no method can beat, and the forest reaches it about as well as the tree does.
   - The forest was *not tuned well* (e.g., m too large so the trees are highly correlated, or trees too deep), so its decorrelation benefit is limited.
   - The training and test sets are quite different (covariate shift), so reductions in training error do not generalize.

7. A boosted model improves validation error for the first 300 trees and then slowly worsens. Explain this pattern and describe how you would choose the number of trees.

   Solution. Early trees reduce *bias* by correcting residual structure that the existing ensemble has not captured; validation error falls. After enough trees, the residuals are mostly noise rather than structure, so subsequent trees fit that noise and *variance* dominates – validation error rises.

This is overfitting, and unlike random forests, boosting can overfit if B is too large.
Choose the number of trees at the *minimum of the validation (or CV) error curve*, ideally with early stopping during fitting. A simpler, slightly more conservative version is the "one-standard-error" rule: pick the smallest B whose CV error is within one SE of the minimum.

8. A neural network has training accuracy of 99% and validation accuracy of 72%. Name two changes you would try first and explain why.

   Solution. The 27-point gap is severe *overfitting*: the network has memorized the training data without generalizing. Any *two* of:

   - *Add regularization:* dropout and/or ℓ_2 weight decay both reduce variance by either randomly dropping units (preventing co-adaptation) or shrinking weights (analogous to ridge).
   - *Early stopping:* stop training at the epoch where validation loss bottoms out, before the network starts memorizing.
   - *Reduce model size:* fewer layers/units lower the parameter count and thus the variance.
   - *Get more data* or use *data augmentation:* more training examples leave less room for memorization.

   The shared logic is that a 27-point gap is a variance problem; the fixes all reduce variance at the cost of some bias.

9. A PCA analysis finds that PC1 explains 70% of the variation in the predictors, but a classifier using PC1 performs poorly. Explain why this is not a contradiction.

   Solution. PCA is *unsupervised* – it finds the direction of maximum variance in X *without ever looking at* Y. There is no guarantee that the direction of maximum variance in the predictors is the direction along which the class labels separate. PC1 could be a "size" or "scale" axis that explains 70% of X's variance but is essentially uncorrelated with Y, while the discriminating direction lies in PC2, PC3, or some combination of later components. "High proportion of variance explained" is a statement about X, not about predictive power for Y.

10. In a dataset with 1% positive cases, compare accuracy, precision, and recall as model evaluation metrics. Which would you emphasize if false negatives are very costly?

    Solution. *Accuracy* is nearly useless here: predicting "negative" for everyone already gets 99% accuracy. *Precision* (TP / (TP + FP)) tells you how trustworthy a positive prediction is. *Recall* (TP / (TP + FN)) tells you what fraction of true positives the model catches.
    If false negatives are very costly (e.g., missing a cancer diagnosis), *emphasize recall*: you cannot afford to miss true positives. You will likely accept lower precision (more false alarms) in exchange. The standard tool to navigate this tradeoff is the precision–recall curve, choosing a threshold that achieves a target recall while keeping precision tolerable. F1 (the harmonic mean of precision and recall) is a useful single-number summary when both matter.

11. Explain why using the test set repeatedly during model development can make the final reported test error too optimistic.

    Solution. Each time you check the test set and adjust the model in response (try a different feature, threshold, hyperparameter), the test set is implicitly being used to *tune* the model –

it stops being independent of your modeling choices. After many such adaptive decisions, your final model has been selected to look unusually good on *that particular* test set, and the reported test error underestimates the true generalization error (a form of optimism / selection bias on the test data). Cross-validation or a separate validation set should be used for iterative tuning, with the test set touched exactly once at the end.

Section 4: Application Questions

1. A **genomics lab** wants to predict disease status from 20,000 gene-expression predictors on 300 patients. (a) Why is OLS a poor choice? (b) Which shrinkage/selection method(s) would you recommend?

   Solution. (a) With $p = 20{,}000 \gg n = 300$, the design matrix has more columns than rows; OLS has *infinitely many* solutions that fit the training data perfectly, and any one of them has astronomical variance and generalizes terribly.
   (b) Use a shrinkage method:

   - *Lasso* is a strong default – the ℓ_1 penalty zeros out most coefficients, identifying a small interpretable subset of candidate genes.
   - *Elastic net* (mix of ℓ_1 and ℓ_2) is often better when groups of genes are highly correlated, since lasso tends to pick one gene from each correlated group somewhat arbitrarily; elastic net keeps groups together.
   - *Ridge* alone gives you stable predictions but no variable selection – useful if you only care about prediction.

   In genomics, lasso or elastic net are typical first choices, with λ tuned by cross-validation. Treat the selected gene list as a *predictive screening*, not a definitive biological causal set.

2. A **bank** chooses between logistic regression and a random forest for loan default, and is required by regulation to **explain individual decisions**. Which should they prefer?

   Solution. *Logistic regression.* It produces a coefficient for every predictor with a clear interpretation ("a 1-unit increase in credit utilization raises the log-odds of default by β, holding others fixed"), so the bank can tell an applicant which factors drove their decision. A random forest is typically more accurate on tabular data, but its predictions come from hundreds of trees interacting; feature importance summarizes the model overall but does not justify any one applicant's score. Choosing the random forest would buy some predictive accuracy at the cost of regulatory compliance and transparency. The bank should accept the accuracy cost unless they can supplement the forest with per-applicant explanation tools (e.g., SHAP values, LIME), which add complexity and require additional auditing.

3. A **real estate analyst** has 80 candidate predictors of house price, many irrelevant or highly correlated, and wants both prediction *and* a short, interpretable list of predictors. Which regularization method would you recommend?

   Solution. *Lasso*, tuned by cross-validation. It simultaneously shrinks coefficients (controlling variance) and performs variable selection (zeroing out the irrelevant ones), so the final model is both predictive and sparse – the analyst gets a handful of predictors rather than all 80. Ridge would deliver similar predictive accuracy but keep every coefficient nonzero, failing the interpretability requirement. If many predictors are strongly correlated and the analyst worries about lasso arbitrarily picking one from each group, *elastic net* is the natural compromise (it keeps correlated groups together while still shrinking and partially selecting).

4. A **labour economist** wants the *causal effect* of education on wages. A friend suggests a deep neural network "because it's the most flexible." Why might a simpler linear regression be better?

   Solution. The economist's goal is not prediction but *identification of a causal parameter*, and flexibility alone does not solve the selection problem. People with more education differ from people with less education in unobserved ways (ability, family background, motivation), and any model – flexible or not – that just correlates education with wages will pick up those confounders. A simpler linear regression – ideally combined with an identification strategy (instrumental variables like compulsory schooling laws, natural experiments, panel design with fixed effects) – delivers a parameter the economist can actually interpret and defend against omitted-variable critiques. A deep network may predict wages better, but it will not tell the economist what happens if we *change* education holding other factors constant.

5. A **startup** with 400 observations is deciding between a large feedforward neural network and a boosted tree ensemble. Which is likely to perform better, and how could the neural net be made to work?

   Solution. The boosted tree ensemble (or random forest) will almost certainly win. Neural networks typically need thousands to millions of observations before their flexibility pays off; with 400 observations, a large network has too many parameters and overfits badly. Tree-based methods are highly competitive on small-to-medium tabular datasets and have well-understood regularization knobs.
   If the startup insists on a neural net, they should: (i) keep the network small (1–2 hidden layers, modest width), (ii) use strong regularization (dropout, ℓ_2 weight decay), (iii) employ early stopping against a validation set, (iv) standardize features, and (v) where possible, use *transfer learning* from a model pre-trained on related data so they are not starting from scratch.

6. A **hospital** uses a boosted tree ensemble to predict 30-day readmission and tells patients: "Your high readmission risk is *caused by* your lab test X and age." What is wrong, and how should they say it?

   Solution. Feature importance from a predictive model is a statement about *correlation within the data used to train the model*, not about causation. Lab test X might be a downstream marker of an underlying condition rather than its cause, and age correlates with many unmeasured things (comorbidities, social support, medication adherence). A more accurate phrasing: "Patients with your profile of lab values and age have historically been *more likely* to be readmitted within 30 days, so you have been flagged as higher risk." This preserves the predictive claim without making a causal claim the model cannot support. Causal claims would require an intervention study or a research design that handles confounding.

7. A **small retailer** wants to quantify **uncertainty** around a regression slope but does not trust the software's normal-theory standard errors. Which Class 9 method could they use?

   Solution. The *bootstrap*. Procedure: draw $B \approx 1000$ samples *with replacement* from the dataset, refit the regression on each resample to get B estimates of the slope $\hat{\beta}_1$, then use the empirical distribution of those estimates:

   - the *bootstrap standard error* is the sample standard deviation of the B slope estimates,

- a *percentile 95% confidence interval* is the interval from the 2.5th to the 97.5th percentile of the bootstrap slopes.

The bootstrap does not assume normally distributed errors, so it gives valid uncertainty estimates even when the textbook formulas would not apply.

8. A **tech company** uses PCA to reduce 50 user-behavior features to 3 components before fitting a classifier. Accuracy *improves*. Why might that happen, and when would PCA before classification *hurt*?

   Solution. *Why it can help:* when the original 50 features are noisy and highly correlated, the first few PCs capture the shared signal across them while most of the noise lands in later components. The classifier then operates on a cleaner, lower-variance representation – effectively a regularization step that reduces overfitting.
   When it hurts: PCA is unsupervised – it optimizes for X-variance with no regard for Y. If the signal useful for predicting Y lies along low-variance directions of X (a small but discriminating signal buried under big but irrelevant variance), PCA will happily discard exactly the components that mattered. *Supervised* dimensionality reduction (e.g., partial least squares, or feature importance from a tree model) avoids this failure mode.

9. A **public health agency** evaluates a model predicting flu outbreaks across 40 small regions. Should they use a validation set, k-fold CV, or LOOCV? Address the bias–variance tradeoff.

   Solution. With only 40 observations, a single *validation set* is a bad idea: the validation set would be tiny (say 10 regions), so the error estimate is very noisy, and the training set is also small.
   LOOCV uses every region as a test point but requires 40 model fits, and – because the 40 training sets overlap heavily (differing by one observation) – the resulting predictions are highly correlated, giving the LOOCV estimate *relatively high variance*. It is also the most expensive option.
   k-fold CV with $k = 5$ or 10 is the sweet spot: lower bias than a single holdout (every fold is used both for training and validation), lower variance than LOOCV (fewer correlated training sets), and tractable to compute.
   Recommendation: 10-fold CV (or 5-fold if computation is tight or rare-event stratification matters).

10. A **sports analytics team** chooses between Lasso regression and a random forest with feature importance to both predict a player's salary and learn *which* statistics matter most. How might the two answers differ?

    Solution. The two methods see "importance" through different lenses, so they can rank predictors differently:

    - *Lasso* tells them which statistics have nonzero *partial linear* relationships with salary after shrinkage. The output is sparse, signed (positive/negative), and easy to communicate.
    - *Random forest feature importance* ranks which statistics contribute most to predictive accuracy across many *nonlinear splits and interactions*. There is no sign, and a variable can be "important" because it matters only for certain subgroups.

The answers can diverge because (i) a statistic with a strong nonlinear or interaction effect may be dropped by lasso but rank highly in the forest, and (ii) among correlated predictors, lasso tends to pick one and zero out the others, while the forest typically spreads importance across them. **Best practice:** look at *both* – the union of important variables is a more honest description of what the data say than either method alone.